Markov chain practice java implement

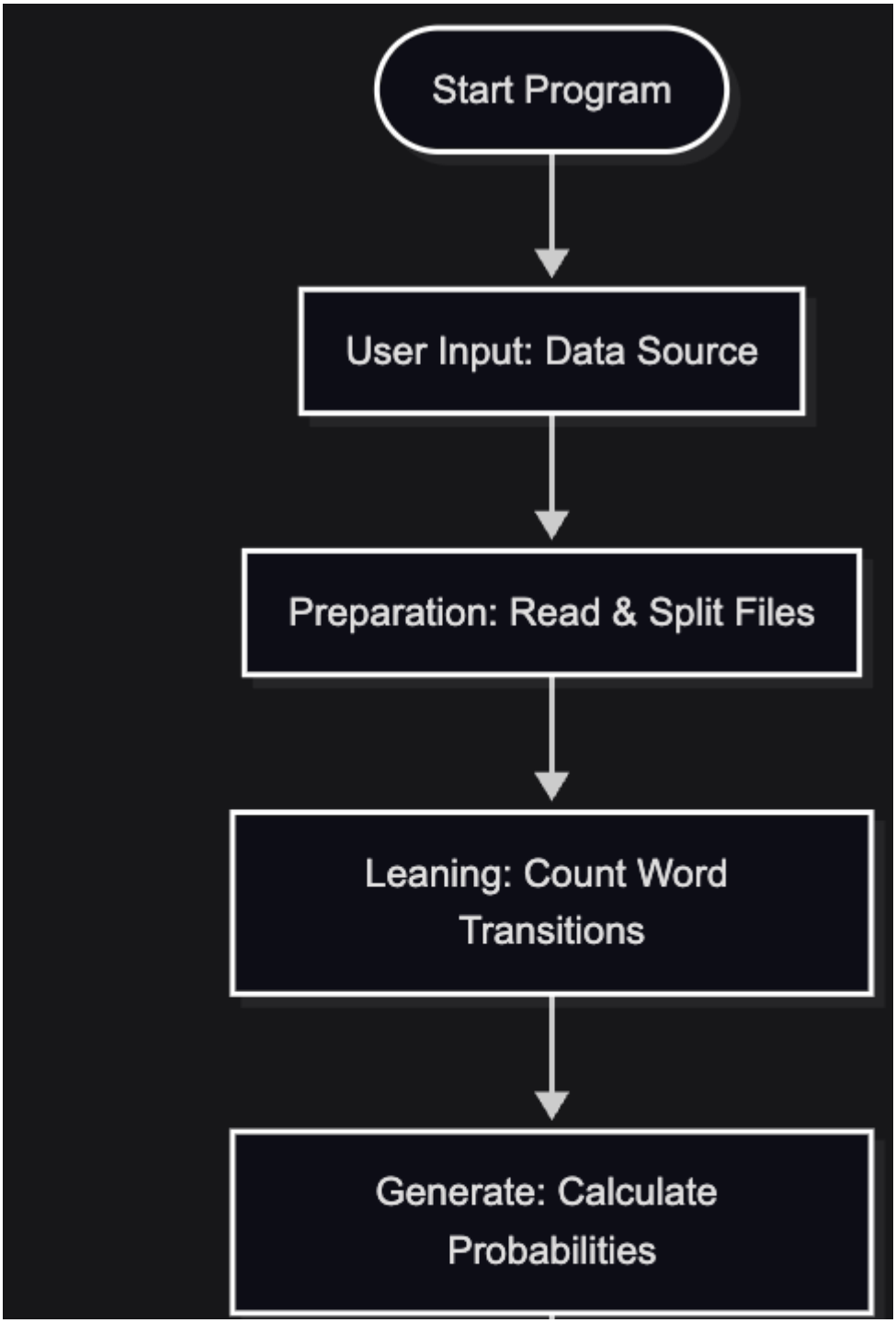
概要

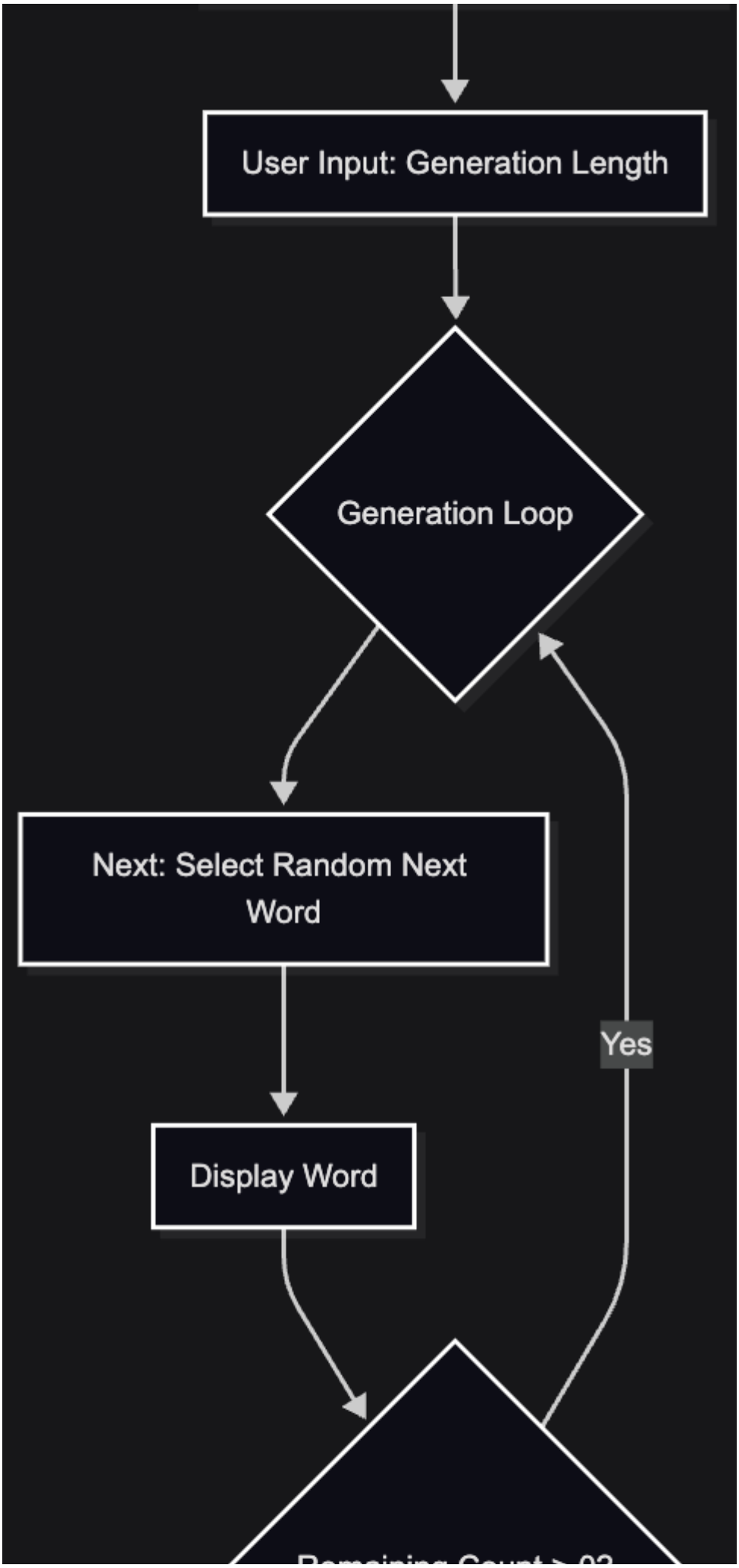
このjavaプロジェクトは任意の英語文字列を学習し、マルコフ連鎖を用いて文を任意の単語数で生成するものです。

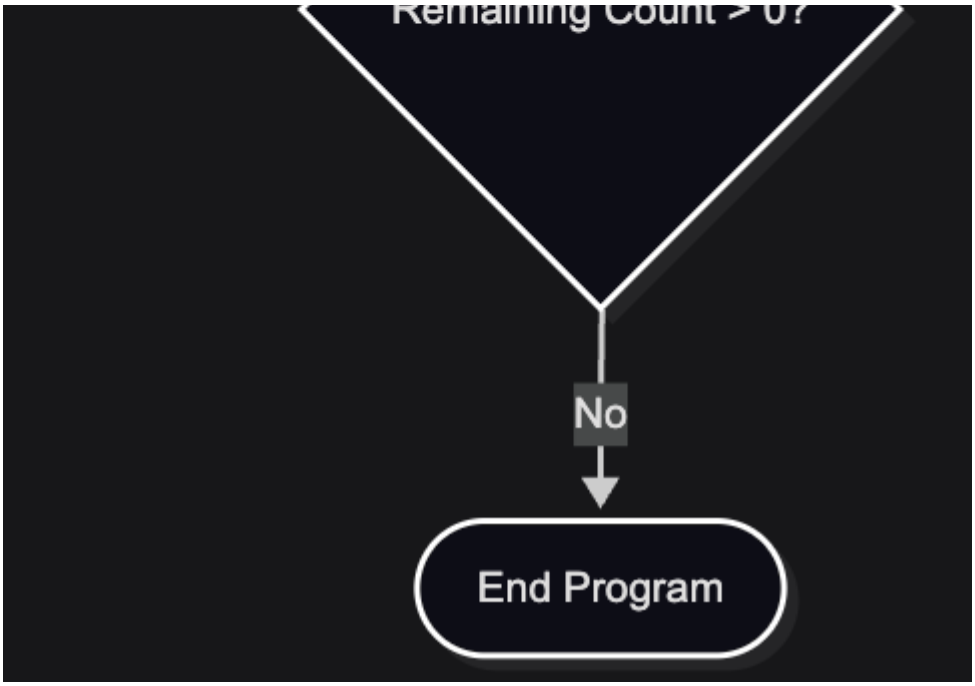
プログラム解説

まず全体は以下のような流れで処理が行われます。以下、ソースコードから一部抜粋しつつ解説を行う。また抜粋の影響でインデントが崩れるが可読性を優先する。

*下記の図はgeminiにソースコードを読ませて生成した図式である。




User Input: Generation Length
Generation Loop
Next: Select Random Next Word
Display Word
Yes



Main.java

7行目から22行目において学習用データの決定を行なっている。デフォルトで用意したセットを使うかどうかをユーザーに選択させるプロンプトを行い、Scannerで結果を取得する。

```
public class Main {
    public static void main(String[] args) {
        // Scannerと全体でのエラーハンドリングのためtry
        try {
            // 学習データのファイルパス指定。デフォルトを使用する場合と自分で用意するか
を選択
            ArrayList<String> lst = new ArrayList<String>();
            System.out.printf("do you use default leaning source? (y/n)>
");
            // 省略
        }
```

23行目から34行目において`Preparation`, `Leaning`, `Generate` Classを用いて、入力データの解析を行なっている。解析についての詳細は各Classの解説を参照。

ここでの工夫点として現在の処理についての説明を出力するようにしている。今回のケースでは処理が高速に終了するため、あまり効果はないが学習データの増大や機能増加によってユーザーの待ち時間が増えた際に、処理が進んでいるのかの確認に効果的であると考えたためである。

```
// Preparationにてデータの取得と単語へのパースを行う
var parsed_data = new Preparation(lst.toArray(new String[0]));

System.out.printf("analyzing...\n");
// Leaningにて各単語の出現回数をカウントしてベクトル的に格納
var leaned_data = new Leaning(parsed_data.get_input());
System.out.printf("analyzed\n");
```

```
// Generateにて出現回数から次単語の出現確率を各ベクトルに変換
System.out.printf("caluculating...\n");
var caled = new Generate(leaned_data);
System.out.printf("caluculated\n");
```

36行目から47行にかけては、実際の出力を行っている。生成する単語数をユーザーに入力してもらい、それに基づいて`Gemerate` Classのメソッドである`loopShow()`を用いて指定回数分の生成と表示を行っている。

```
// 何単語分生成するかを質問
System.out.printf("\nhow long do you want to generate?>");
var i = sc.nextInt();
// 入力はこれ以降ないのでここでclose()
sc.close();

// 出力のセパレータ
System.out.printf("\n");

// スタートはTheで固定
// iは生成する単語数
caled.loopShow("The", i);
```

50行目はエラーハンドリングとして上記の処理で発生した例外をすべてcatchしてそのままエラーとして表示している。今回はリトライで成功すするような処理はないため、そのままExit(1)として終了する。

```
catch (Exception e)
```

56行目以降ではデフォルトの学習データのパスの定義などを行っている。現在は私の環境でしか動作しないため再現性を高めるためにはもっと汎用的なパスの生成を行う必要がある。

```
static String getTestDataSetPath()
```

Preparation.java

パスから実際のファイルの内容の取得と、実際の解析前の文字列の単語分割を行う。

Preparationのコンストラクタ。`get_input()`と`get_some_input()`をまとめて実行する実質的なラッパー関数。

```
public Preparation(String[] p) throws Exception {
    try {
        this.input = this.Parse(this.get_some_input(p));
        System.out.printf("loaded %d words\n\n", this.input.length);
```

```
        } catch (Exception e) {
            throw e;
        }
    }
```

1つのファイルの内容を取得する関数。ファイルの存在確認も行う。

```
    String get_input(String path) throws Exception {
        var p = Paths.get(path);

        // ファイルの存在確認
        if (!Files.isRegularFile(p)) {
            throw new InvalidInput(String.format("invalid input: %s", p));
        }

        // Io操作に失敗する可能性があるため
        try {
            return Files.readString(p).replace(".", "");
        } catch (Exception e) {
            throw e;
        }
    }
```

`get_input()`を用いて、複数のファイルをパスのリストから取得する関数。これにより簡単に複数のファイルから学習を行える。

```
    String get_some_input(String[] pathes) throws Exception {
        String res = new String();
        for (var p : pathes) {
            res += get_input(p);
        }

        return res;
    }
```

シンプルにStringのメソッドでスペース区切りにしてパースを行う関数。日本語対応などを行うならばらここで文法的な解析を行うなど、モジュール性を意識した。

```
    String[] Parse(String input) {
        return input.split(" ");
    }
```

Leaning.java

`HashMap<現在の単語, HashMap<次の単語, 出現回数>>`となるようにカウントを行った。

```
HashMap<String, HashMap<String, Integer>> WordsMap;
```

全体的な工夫点

- 処理のレイヤーを細かく分けることで今後の変更を行いやすくなった
  - 例えば単語の分割方法を変更する際には`Preparation.java`の処理変更すれば問題なく更新可能
- データ構造に`HashMap`を用いることで高速な処理を行えるようにした